Projeto Final: Architecture Challenge

Atividade em Grupo de 4/5 alunos.
O projeto final tem como objetivo consolidar os conhecimentos adquiridos na disciplina de Projetos de Arquitetura de Sistemas, aplicando os princípios de **SOLID**, **GRASP**, **Design Patterns** e **Arquitetura de Microserviços** na construção de um *back-end* robusto desenvolvido a partir do cenário escolhidos e trabalhados pela equipe na atividade de padrões de projetos. Dessa forma, o objetivo não é criar um novo sistema, mas evoluir e aprofundar a solução já desenvolvida, aplicando os conhecimentos adquiridos ao longo da disciplina de Projetos de Arquitetura de Sistemas.

I. Definição do Sistema e Escopo do Projeto

Cada equipe deverá utilizar o sistema anteriormente desenvolvido como base para a construção do projeto final, podendo realizar as adaptações, extensões e refatorações necessárias para atender aos requisitos desta etapa.

O projeto deverá demonstrar a aplicação prática dos seguintes conceitos:

- **Princípios SOLID;**
- **GRASP;**
- **Design Patterns;**
- **Arquitetura de Microserviços;**
- **Desenvolvimento de APIs REST;**
- **Boas práticas de organização, modularização e manutenção de sistemas.**

A equipe deverá identificar, dentro do sistema escolhido anteriormente, quais funcionalidades e regras de negócio são mais adequadas para serem exploradas no projeto final. O escopo deverá ser suficientemente amplo para permitir a aplicação dos conceitos arquiteturais estudados, mas compatível com o prazo e o tamanho da equipe.

Evolução do Sistema

O projeto final deverá representar uma evolução significativa do sistema desenvolvido na atividade anterior. Não será suficiente apenas reproduzir as funcionalidades já existentes.

A equipe deverá:

1. **Reaproveitar o domínio, cenário e principais regras de negócio definidos na atividade anterior;**
2. **Identificar pontos do sistema que possam ser aprimorados por meio dos princípios e padrões estudados;**
3. **Expandir ou refatorar funcionalidades existentes;**
4. **Definir uma arquitetura adequada para o back-end;**
5. **Identificar possíveis limites de responsabilidade e separar os componentes do sistema em serviços;**
6. **Implementar os serviços utilizando Spring Boot/Java;**
7. **Disponibilizar as funcionalidades por meio de APIs REST;**
8. **Justificar as principais decisões arquiteturais e de projeto adotadas.**

Arquitetura de Microserviços

A equipe deverá propor uma divisão do sistema em microserviços, considerando as responsabilidades e os limites de cada parte do domínio.

A quantidade e a responsabilidade dos microserviços não serão previamente definidas, pois deverão ser determinadas de acordo com as características do sistema escolhido pela equipe. Porém devem possuir no **mínimo 4 microserviços**.

Por exemplo, dependendo do domínio desenvolvido anteriormente, a equipe poderá identificar serviços relacionados a:

- Usuários e autenticação;
- Produtos ou catálogo;
- Pedidos ou transações;
- Estoque;
- Pagamentos;
- Recursos;
- Relatórios;
- Processamento de regras de negócio;
- Ou outras responsabilidades específicas do domínio escolhido.

A divisão proposta deverá ser justificada pela equipe, demonstrando como os princípios de coesão, baixo acoplamento, separação de responsabilidades e demais conceitos de arquitetura foram considerados.

Front-end

A tecnologia de front-end é livre, podendo ser utilizadas tecnologias como React, Vue, Angular, JavaScript puro, entre outras.

O front-end deverá consumir as APIs REST disponibilizadas pelo back-end e demonstrar, de forma funcional, as principais operações e regras de negócio implementadas no projeto final.

II. Requisitos Técnicos e Arquiteturais (Obrigatórios)

Tópico	Requisito Mínimo	Detalhes da Aplicação
Arquitetura Geral	**Microserviços**	O sistema **deve** ser dividido em, no mínimo, **4 (quatro)** microserviços independentes.

Framework & Linguagem	**Spring Boot (Java)**	O *back-end* de todos os microserviços deve ser implementado usando Java e o *framework* Spring Boot.
Padrões de Projeto	**9 Padrões (3 de cada Categoria)**	A equipe deve utilizar e documentar o uso de **3 Padrões Criacionais**, **3 Padrões Estruturais** e **3 Padrões Comportamentais**.
Princípios de Design	**SOLID e GRASP**	A aplicação dos princípios (ex: Alta Coesão, Baixo Acoplamento, Princípio Aberto/Fechado) deve ser evidente e justificada.
Arquitetura Interna	**MVC ou Serviços**	A equipe pode optar pela estrutura MVC ou por uma arquitetura em camadas baseada em Serviços (Service Layer) dentro de cada microserviço.
Front-end	**Livre**	A tecnologia de *front-end* é livre (React, Vue, Angular, JavaScript puro, etc.), contanto que consuma as APIs REST do *back-end*.

Etapas:

Entrega 1: Plano de Trabalho

Entrega 2: Apresentação Final

Entrega 1: Arquitetura, Design e Primeira Apresentação

1. **Descrição da Arquitetura**
    - Deverá ser utilizada obrigatoriamente uma arquitetura baseada em Microserviços, contemplando no mínimo 4 (quatro) microserviços independentes.
    - A equipe deve modelar a arquitetura do sistema, escolher um padrão arquitetural adequado, podendo escolher: Modelo em Camadas ou MVC; e justificar essa escolha.
    - Devem ser apresentados **diagramas arquiteturais** e a divisão de **módulos do sistema**.

- Utilização de **Spring Boot em Java** para o *back-end*.

2. **Padrões de Projeto (Design Patterns):**
   - A equipe deverá identificar e aplicar pelo menos **9 padrões de projeto**: 3 Criacionais, 3 Estruturais e 3 Comportamentais.
     - O mínimo exigido é de 3 padrões por categorias.
   - A escolha de cada padrão deve ser justificada, mostrando em qual parte do sistema ele será implementado.
3. **Outros Requisitos**
   - A aplicação dos princípios **SOLID e GRASP** deve ser evidente e justificada no código.
4. **Divisão de Funcionalidades entre a Equipe:**
   - Cada membro da equipe terá uma responsabilidade clara na arquitetura e na implementação:
     - **Membro 1:** Responsável pela **definição e modelagem da arquitetura** (diagrama de classes, sequência e a escolha do padrão arquitetural).
     - **Membro 2:** Implementação de **padrões estruturais** e a camada de persistência (ex: uso do padrão **DAO** ou **Repository**).
     - **Membro 3:** Implementação de **padrões comportamentais** e controle da interação do sistema (ex: **Observer** ou **Strategy**).
     - **Membro 4 (opcional):** Responsável pela **integração de APIs** ou funcionalidades adicionais (ex: autenticação, relatórios).

5. **Primeira Apresentação:**
   - **A equipe deve realizar uma apresentação explicando:**
     - O tema do sistema escolhido e suas funcionalidades principais.
     - A **arquitetura definida** mostrando os padrões arquiteturais e as decisões técnicas (Detalhamento com o ADR).
     - Como os padrões de projeto serão aplicados no sistema, com exemplos de código e diagramas UML (Caso de uso, diagrama de classes e sequência).
     - **Diagrama de Arquitetura**: MDL (https://mdlmodel.com/)
     - Utilização de Software/Plataforma de Gerenciamento de projetos (Trello, Notion, Asana…).
     - Apresentação de Backlog das Atividades
     - Aplicação da arquitetura de microserviços para estruturação dos serviços.
   - A apresentação será avaliada com base na **clareza da explicação**, **justificativa das escolhas arquiteturais** e **uso dos padrões de projeto**.

Entrega 2: Desenvolvimento, Integração e Apresentação Final

1. **Desenvolvimento Completo do Sistema:**
   - Implementar o sistema com base no design e na arquitetura propostos na Etapa 1, garantindo que as funcionalidades previstas sejam implementadas corretamente.

    - Cada membro deve implementar sua parte do sistema conforme as responsabilidades definidas anteriormente.
2. **Testes e Integração:**
    - Realizar testes unitários e de integração para garantir que as partes do sistema funcionem corretamente e que a integração entre elas seja estável.
    - A equipe deverá demonstrar o uso de ferramentas de testes automatizados e explicar como os testes foram conduzidos.
3. **Documentação Técnica:**
    - A equipe deve fornecer uma documentação detalhada, incluindo os seguintes pontos:
        - Descrição da arquitetura final.
        - Explicação dos padrões de projeto implementados.
        - Diagramas UML atualizados, conforme o desenvolvimento avançou.
        - Justificativas das decisões técnicas que foram tomadas ao longo do desenvolvimento.
4. **Apresentação Final:**
    - Na apresentação final, a equipe deve demonstrar:
        - O sistema em funcionamento, explicando suas principais funcionalidades.
        - O cumprimento dos requisitos definidos na Etapa 1, mostrando como os padrões de projeto foram aplicados no código.
        - Testes realizados e os resultados obtidos.
        - A avaliação levará em conta a **coerência entre o planejamento inicial e a implementação**, **qualidade da entrega** e **clareza da explicação**.

Critérios de Avaliação:

Primeira Entrega: Arquitetura, Design e Apresentação (E-1) Pontuação: 1.5

1. **Modelagem da Arquitetura: (Pt: 0.4)**
    - Qualidade da modelagem e justificativa para a escolha do padrão arquitetural (Monolítico, Microsserviços, Serverless, etc.).
    - Clareza e detalhamento dos diagramas arquiteturais (ex: diagrama de classes e sequência).
    -
2. **Aplicação de Padrões de Projeto: (Pt: 0.3)**
    - Identificação correta e coerente dos padrões de projeto a serem aplicados.
    - Justificativa clara da escolha de cada padrão e o local onde ele será implementado.
3. **Participação Individual e Colaboração em Equipe: (Pt: 0.3)**

    - Avaliação da divisão de responsabilidades entre os membros e a contribuição individual de cada um.
4. **Qualidade da Apresentação: (Pt: 0.5)**
    - Clareza e organização da apresentação.
    - Capacidade da equipe de justificar suas escolhas técnicas e responder a perguntas.

Segunda Entrega: Desenvolvimento, Integração e Apresentação Final (E-2) Pontuação: 2.5

1. **Funcionamento do Sistema: (Pt: 0.7)**
    - O sistema atende aos requisitos funcionais definidos na Etapa 1?
    - O sistema está funcional, integrado e bem testado?
2. **Aplicação e Integração de Padrões: (Pt: 0.5)**
    - Verificar se os padrões de projeto foram implementados corretamente e de forma coerente.
    - Qualidade da modularização do sistema e sua arquitetura.
3. **Qualidade do Código e Implementação: (Pt: 0.5)**
    - O código segue boas práticas de desenvolvimento? Está bem estruturado, modular e de fácil manutenção?
    - A equipe realizou testes adequados e demonstrou o uso de ferramentas de testes automatizados?
4. **Documentação e Apresentação Final: (Pt: 0.8)**
    - A documentação técnica está completa e detalhada?
    - A apresentação final demonstra o funcionamento completo do sistema e o uso correto dos padrões de projeto e da arquitetura definida?

PONTUAÇÃO TOTAL

Etapa	Critério	Descrição	Peso
E-1 – Plano de Trabalho + MVP	Arquitetura e Modelagem	Qualidade da arquitetura proposta, divisão dos microserviços, diagramas e justificativa das decisões arquiteturais.	4%
	Design Patterns	Identificação e planejamento dos padrões de projeto, contemplando 3 padrões criacionais, 3 estruturais e 3 comportamentais, com justificativa de aplicação.	3%
	SOLID e GRASP	Identificação dos princípios que serão aplicados e demonstração de como contribuirão para a organização do sistema.	2%

	Plano de Trabalho e Backlog	Organização das atividades, divisão de responsabilidades entre os integrantes, cronograma e utilização de ferramenta de gerenciamento de projetos.	2%
	MVP	Implementação de uma versão inicial funcional do sistema, demonstrando a viabilidade da arquitetura e das principais funcionalidades.	2%
	Apresentação e Justificativas	Clareza na apresentação, domínio do projeto e capacidade de justificar as decisões técnicas e arquiteturais.	2%
	Subtotal E-1		**15%**
E-2 – Apresentação Final	Funcionamento e Integração	Sistema funcionando de acordo com os requisitos definidos, com integração adequada entre os microserviços e front-end.	6%
	Arquitetura de Microserviços	Implementação da arquitetura proposta, comunicação entre serviços, separação de responsabilidades, coesão e baixo acoplamento.	4%
	Design Patterns	Implementação correta e coerente dos 9 padrões de projeto previstos, demonstrando sua utilização no código e no domínio do sistema.	4%
	SOLID, GRASP e Qualidade do Código	Aplicação dos princípios de design, organização, modularização, legibilidade, manutenibilidade e qualidade da implementação.	3%
	Testes e Qualidade	Testes unitários e de integração, utilização de ferramentas automatizadas e demonstração dos resultados.	3%
	Documentação Técnica	Documentação da arquitetura final, padrões utilizados, diagramas atualizados e ADRs/justificativas das decisões técnicas.	2%
	Apresentação e Demonstração	Clareza da apresentação, demonstração do sistema, capacidade de explicar a implementação e responder aos questionamentos.	3%
	Subtotal E-2		**25%**

	TOTAL DO PROJETO FINAL		**40%**